APPOINTMENT BOOKING SYSTEM

Complete Backend API Documentation

Node.js + Express + MongoDB

A Comprehensive Learning Resource for Students

Generated: 27/4/2026

TABLE OF CONTENTS

1. Project Overview
2. Technology Stack
3. Project Architecture
4. Database Schema
5. Authentication Module
6. Users Management Module
7. Specialists Module
8. Doctors Module
9. Slots Module
10. Appointments Module
11. Payments Module
12. Real-time Video Calls Module
13. Real-time Chat Module
14. Error Handling & Response Format
15. Security Best Practices
16. API Testing Examples

1. PROJECT OVERVIEW

This is a complete appointment booking system backend built with Node.js, Express.js, and MongoDB. The system enables users to book medical appointments with doctors, make real-time video calls, and communicate through chat.

Key Features:

' User authentication with email and phone number
' Doctor and Specialist management
' Appointment booking with available slots
' Payment integration for appointment fees
' Real-time video/audio calling using WebRTC
' Real-time chat messaging system
' Complete CRUD operations for all resources
' JWT-based authentication
' MongoDB database with Mongoose ODM

Target Users:

• Patients: Book appointments with doctors
• Doctors: Manage availability and appointments
• Administrators: Manage specialists and doctors

2. TECHNOLOGY STACK

Backend Framework:

• Express.js v5.2.1 - Web framework for routing and middleware

Database:

• MongoDB - NoSQL document database
• Mongoose v9.5.0 - ODM for MongoDB with schema validation

Authentication:

• bcryptjs v3.0.3 - Password hashing
• jsonwebtoken v9.0.3 - JWT generation and verification

Real-time Communication:

• Socket.io v4.8.3 - Bidirectional real-time communication
• WebRTC - Peer-to-peer video/audio streaming

File Handling:

• multer v2.1.1 - File upload middleware
• express-fileupload v1.5.2 - Alternative file upload
• pdfkit - PDF generation

Utilities:

• dotenv v17.4.2 - Environment variable management
• cors v2.8.6 - Cross-origin resource sharing

3. PROJECT ARCHITECTURE

Directory Structure:

```
my-appointment-app/
% % %  server.js                 # Main server file
% % %  .env                      # Environment variables
% % %  src/
%    % % %  app.js               # Express app configuration
%    % % %  config/
%    %    % % %  db.js            # Database connection
%    % % %  models/
%    %    % % %  User.js
%    %    % % %  Doctor.js
%    %    % % %  Specialist.js
%    %    % % %  Appointment.js
%    %    % % %  Slot.js
%    %    % % %  Chat.js
%    %    % % %  Payment.js
%    % % %  controllers/
%    %    % % %  authController.js
%    %    % % %  specialist.controller.js
%    %    % % %  doctor.controller.js
%    %    % % %  appointment.controller.js
%    %    % % %  slot.controller.js
%    %    % % %  payment.controller.js
%    %    % % %  chat.controller.js
%    %    % % %  videoCall.controller.js
%    % % %  routes/
%    %    % % %  authRoutes.js
%    %    % % %  specialistRoutes.js
%    %    % % %  doctorRoutes.js
%    %    % % %  appointmentRoutes.js
%    %    % % %  slotRoutes.js
%    %    % % %  paymentRoutes.js
%    %    % % %  chatRoutes.js
%    %    % % %  videoCallRoutes.js
%    % % %  middleware/
%    %    % % %  auth.js          # JWT authentication middleware
%    % % %  sockets/
%        % % %  videoCallSocket.js
%        % % %  chatSocket.js
% % %  public/
    % % %  video-call.html      # Test client
```

Request Flow:

Client Request !’ Express Middleware !’ Auth Middleware !’ Controller !’ Database !’ Response

Architecture Pattern:

The application follows the MVC (Model-View-Controller) pattern with separation of concerns:
• Models: Database schemas and relationships
• Controllers: Business logic and request handling
• Routes: API endpoint definitions
• Middleware: Authentication and request processing

4. DATABASE SCHEMA

User Model

```
{
  _id: ObjectId,
  name: String (required),
  email: String (unique, required if no phone),
  phone: String (unique, default: ""),
  password: String (hashed, required),
  role: String (enum: ["patient", "doctor"], default: "patient"),
  profilePic: String (optional),
  dateOfBirth: Date,
  gender: String (enum: ["male", "female", "other"]),
  address: String,
  createdAt: DateTime (auto),
  updatedAt: DateTime (auto)
}
```

Doctor Model

```
{
  _id: ObjectId,
  userId: ObjectId (references User),
  specialistId: ObjectId (references Specialist),
  licenseNumber: String (required, unique),
  experience: Number (years),
  qualifications: [String],
  consultationFee: Number,
  biography: String,
  workingHours: {
    monday: { start: Time, end: Time },
    // ... other days
  },
  isActive: Boolean (default: true),
  createdAt: DateTime,
  updatedAt: DateTime
}
```

Specialist Model

```
{
  _id: ObjectId,
  name: String (required, unique),
  description: String,
  icon: String (optional),
  isActive: Boolean (default: true),
  createdAt: DateTime,
  updatedAt: DateTime
}
```

Appointment Model

```
{
  _id: ObjectId,
  patientId: ObjectId (references User),
  doctorId: ObjectId (references Doctor),
  slotId: ObjectId (references Slot),
  status: String (enum: ["pending", "confirmed", "completed", "cancelled"]),
  appointmentDate: DateTime,
  notes: String,
  paymentStatus: String (enum: ["unpaid", "paid"]),
  videoCallUrl: String (optional),
  createdAt: DateTime,
  updatedAt: DateTime
}
```

Slot Model

```
{
  _id: ObjectId,
  doctorId: ObjectId (references Doctor),
  startTime: DateTime (required),
  endTime: DateTime (required),
  isAvailable: Boolean (default: true),
  createdAt: DateTime,
  updatedAt: DateTime
}
```

Chat Model

```
{
  _id: ObjectId,
  senderId: ObjectId (references User),
  recipientId: ObjectId (references User),
  appointmentId: ObjectId (references Appointment, optional),
  message: String (required),
  attachmentUrl: String (optional),
  attachmentType: String (optional),
  isRead: Boolean (default: false),
  readAt: DateTime,
  deletedBySender: Boolean (default: false),
  deletedByRecipient: Boolean (default: false),
  createdAt: DateTime,
  updatedAt: DateTime
}
```

Payment Model

```
{
  _id: ObjectId,
  appointmentId: ObjectId (references Appointment),
  userId: ObjectId (references User),
  amount: Number,
  currency: String (default: "USD"),
  status: String (enum: ["pending", "completed", "failed"]),
  paymentMethod: String (enum: ["card", "paypal", "bank_transfer"]),
  transactionId: String,
  createdAt: DateTime,
  updatedAt: DateTime
}
```

5. AUTHENTICATION MODULE

Overview

The authentication module handles user registration, login, and JWT token management. Users can authenticate using either email or phone number.

5.1 REGISTER USER

Endpoint: POST /api/auth/register

Request Format:

```
{
  "name": "John Doe",
  "email": "john@example.com",      // Optional if phone provided
  "phone": "+1234567890",           // Optional if email provided
  "password": "securePassword123",
  "role": "patient"                 // "patient" or "doctor"
}
```

Response Format (Success - 201):

```
{
  "success": true,
  "message": "User registered successfully",
  "user": {
    "_id": "507f1f77bcf86cd799439011",
    "name": "John Doe",
    "email": "john@example.com",
    "phone": "+1234567890",
    "role": "patient",
    "createdAt": "2026-04-27T10:30:00Z"
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Response Format (Error - 400):

```
{
  "success": false,
  "message": "Email already exists or provide email or phone"
}
```

5.2 LOGIN USER

Endpoint: POST /api/auth/login

Request Format:

```
{
  "email": "john@example.com",      // OR use "phone" instead
  "password": "securePassword123"
}
```

Response Format (Success - 200):

```
{
  "success": true,
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "_id": "507f1f77bcf86cd799439011",
    "name": "John Doe",
    "role": "patient"
  }
}
```

Response Format (Error - 401):

```
{
  "success": false,
```

```
  "message": "Invalid email/phone or password"
}
```

Implementation Details:

• Passwords are hashed using bcryptjs with 10 salt rounds
• JWT tokens expire in 7 days
• Both email and phone fields are unique in the database
• At least one (email or phone) must be provided during registration
• JWT token should be sent in Authorization header: "Bearer <token>"

6. USERS MANAGEMENT MODULE

6.1 GET ALL USERS (Admin Only)

Endpoint: GET /api/users

Query Parameters:

```
?role=patient&page=1&limit=10&search=john
```

Response Format (200):

```
{
  "success": true,
  "users": [
    {
      "_id": "507f1f77bcf86cd799439011",
      "name": "John Doe",
      "email": "john@example.com",
      "phone": "+1234567890",
      "role": "patient",
      "dateOfBirth": "1990-05-15",
      "gender": "male",
      "address": "123 Main St, City"
    }
  ],
  "total": 50,
  "page": 1,
  "pages": 5
}
```

6.2 GET USER PROFILE

Endpoint: GET /api/users/profile
Headers: Authorization: Bearer <token>

Response Format (200):

```
{
  "success": true,
  "user": {
    "_id": "507f1f77bcf86cd799439011",
    "name": "John Doe",
    "email": "john@example.com",
    "phone": "+1234567890",
    "role": "patient",
    "profilePic": "url_to_image",
    "dateOfBirth": "1990-05-15",
    "gender": "male",
    "address": "123 Main St, City"
  }
}
```

6.3 UPDATE USER PROFILE

Endpoint: PUT /api/users/profile
Headers: Authorization: Bearer <token>

Request Format:

```
{
  "name": "Jane Doe",
  "dateOfBirth": "1990-05-15",
  "gender": "female",
  "address": "456 Oak Ave, City",
  "profilePic": "base64_or_url"
}
```

Response Format (200):

```
{
```

```
  "success": true,
  "message": "Profile updated successfully",
  "user": { ... updated user object ... }
}
```

6.4 DELETE USER

Endpoint: DELETE /api/users/:userId
Headers: Authorization: Bearer <token>

Response Format (200):

```
{
  "success": true,
  "message": "User deleted successfully"
}
```

7. SPECIALISTS MODULE

Specialists are medical specializations (e.g., Cardiology, Neurology, etc.). Doctors are associated with specialists.

7.1 GET ALL SPECIALISTS

Endpoint: GET /api/specialists

Response Format (200):

```
{
  "success": true,
  "specialists": [
    {
      "_id": "507f1f77bcf86cd799439012",
      "name": "Cardiology",
      "description": "Heart and cardiovascular diseases",
      "icon": "url_to_icon",
      "isActive": true,
      "createdAt": "2026-04-01T10:00:00Z"
    },
    {
      "_id": "507f1f77bcf86cd799439013",
      "name": "Neurology",
      "description": "Nervous system disorders",
      "icon": "url_to_icon",
      "isActive": true,
      "createdAt": "2026-04-01T10:00:00Z"
    }
  ]
}
```

7.2 GET SPECIALIST BY ID

Endpoint: GET /api/specialists/:id

Response Format (200):

```
{
  "success": true,
  "specialist": {
    "_id": "507f1f77bcf86cd799439012",
    "name": "Cardiology",
    "description": "Heart and cardiovascular diseases",
    "icon": "url_to_icon",
    "isActive": true,
    "createdAt": "2026-04-01T10:00:00Z"
  }
}
```

7.3 CREATE SPECIALIST (Admin Only)

Endpoint: POST /api/specialists
Headers: Authorization: Bearer <admin_token>

Request Format:

```
{
  "name": "Cardiology",
  "description": "Heart and cardiovascular diseases",
  "icon": "url_to_icon"
}
```

Response Format (201):

```
{
  "success": true,
  "message": "Specialist created successfully",
  "specialist": { ... specialist object ... }
```

```
}
```

7.4 UPDATE SPECIALIST

Endpoint: PUT /api/specialists/:id
Headers: Authorization: Bearer <admin_token>

Request Format:

```
{
  "name": "Updated Specialist Name",
  "description": "Updated description",
  "isActive": true
}
```

Response Format (200):

```
{
  "success": true,
  "message": "Specialist updated successfully",
  "specialist": { ... updated specialist object ... }
}
```

7.5 DELETE SPECIALIST

Endpoint: DELETE /api/specialists/:id
Headers: Authorization: Bearer <admin_token>

Response Format (200):

```
{
  "success": true,
  "message": "Specialist deleted successfully"
}
```

8. DOCTORS MODULE

The Doctors module manages doctor profiles, specializations, and availability.

8.1 GET ALL DOCTORS

Endpoint: GET /api/doctors

Query Parameters:

```
?specialistId=507f...&page=1&limit=10
```

Response Format (200):

```
{
  "success": true,
  "doctors": [
    {
      "_id": "507f1f77bcf86cd799439014",
      "userId": {
        "_id": "507f1f77bcf86cd799439011",
        "name": "Dr. John Smith",
        "email": "john.smith@hospital.com"
      },
      "specialistId": {
        "_id": "507f1f77bcf86cd799439012",
        "name": "Cardiology"
      },
      "licenseNumber": "LICENSE12345",
      "experience": 10,
      "qualifications": ["MBBS", "MD"],
      "consultationFee": 50,
      "biography": "Experienced cardiologist with 10 years...",
      "isActive": true
    }
  ],
  "total": 25,
  "page": 1,
  "pages": 3
}
```

8.2 GET DOCTOR BY ID

Endpoint: GET /api/doctors/:id

Response Format (200):

```
{
  "success": true,
  "doctor": {
    "_id": "507f1f77bcf86cd799439014",
    "userId": { ... user details ... },
    "specialistId": { ... specialist details ... },
    "licenseNumber": "LICENSE12345",
    "experience": 10,
    "qualifications": ["MBBS", "MD"],
    "consultationFee": 50,
    "biography": "Experienced cardiologist...",
    "workingHours": {
      "monday": { "start": "09:00", "end": "17:00" },
      "tuesday": { "start": "09:00", "end": "17:00" }
      // ... other days
    },
    "isActive": true
  }
}
```

8.3 CREATE DOCTOR

Endpoint: POST /api/doctors
Headers: Authorization: Bearer <admin_token>

Request Format:

```
{
  "userId": "507f1f77bcf86cd799439011",
  "specialistId": "507f1f77bcf86cd799439012",
  "licenseNumber": "LICENSE12345",
  "experience": 10,
  "qualifications": ["MBBS", "MD"],
  "consultationFee": 50,
  "biography": "Experienced cardiologist...",
  "workingHours": {
    "monday": { "start": "09:00", "end": "17:00" },
    "tuesday": { "start": "09:00", "end": "17:00" }
  }
}
```

Response Format (201):

```
{
  "success": true,
  "message": "Doctor created successfully",
  "doctor": { ... doctor object ... }
}
```

8.4 UPDATE DOCTOR

Endpoint: PUT /api/doctors/:id
Headers: Authorization: Bearer <token>

Request Format:

```
{
  "consultationFee": 60,
  "biography": "Updated biography...",
  "workingHours": { ... updated hours ... }
}
```

Response Format (200):

```
{
  "success": true,
  "message": "Doctor updated successfully",
  "doctor": { ... updated doctor object ... }
}
```

8.5 DELETE DOCTOR

Endpoint: DELETE /api/doctors/:id
Headers: Authorization: Bearer <admin_token>

Response Format (200):

```
{
  "success": true,
  "message": "Doctor deleted successfully"
}
```

9. SLOTS MODULE

Slots represent available time periods when a doctor can see patients. Patients book appointments by selecting available slots.

9.1 GET AVAILABLE SLOTS

Endpoint: GET /api/slots/doctor/:doctorId

Query Parameters:

```
?date=2026-05-15&available=true
```

Response Format (200):


```
{
  "success": true,
  "slots": [
    {
      "_id": "507f1f77bcf86cd799439015",
      "doctorId": "507f1f77bcf86cd799439014",
      "startTime": "2026-05-15T09:00:00Z",
      "endTime": "2026-05-15T09:30:00Z",
      "isAvailable": true
    },
    {
      "_id": "507f1f77bcf86cd799439016",
      "doctorId": "507f1f77bcf86cd799439014",
      "startTime": "2026-05-15T09:30:00Z",
      "endTime": "2026-05-15T10:00:00Z",
      "isAvailable": true
    }
  ]
}
```


9.2 GET SLOT BY ID

Endpoint: GET /api/slots/:id

Response Format (200):


```
{
  "success": true,
  "slot": {
    "_id": "507f1f77bcf86cd799439015",
    "doctorId": "507f1f77bcf86cd799439014",
    "startTime": "2026-05-15T09:00:00Z",
    "endTime": "2026-05-15T09:30:00Z",
    "isAvailable": true,
    "createdAt": "2026-04-20T10:00:00Z"
  }
}
```


9.3 CREATE SLOT

Endpoint: POST /api/slots
Headers: Authorization: Bearer <doctor_token>

Request Format:


```
{
  "doctorId": "507f1f77bcf86cd799439014",
  "startTime": "2026-05-15T09:00:00Z",
  "endTime": "2026-05-15T09:30:00Z"
}
```


Response Format (201):


```
{
  "success": true,
  "message": "Slot created successfully",
```

```
  "slot": { ... slot object ... }
}
```

9.4 UPDATE SLOT (PATCH)

Endpoint: PATCH /api/slots/:id
Headers: Authorization: Bearer <doctor_token>

Request Format:

```
{
  "isAvailable": false
}
```

Response Format (200):

```
{
  "success": true,
  "message": "Slot updated successfully",
  "slot": { ... updated slot object ... }
}
```

9.5 DELETE SLOT

Endpoint: DELETE /api/slots/:id
Headers: Authorization: Bearer <doctor_token>

Response Format (200):

```
{
  "success": true,
  "message": "Slot deleted successfully"
}
```

10. APPOINTMENTS MODULE

The Appointments module manages the booking and tracking of patient appointments with doctors.

10.1 GET ALL APPOINTMENTS

Endpoint: GET /api/appointments
Headers: Authorization: Bearer <token>

Query Parameters:

```
?status=confirmed&doctorId=507f...&page=1&limit=10
```

Response Format (200):

```
{
  "success": true,
  "appointments": [
    {
      "_id": "507f1f77bcf86cd799439017",
      "patientId": { ... patient details ... },
      "doctorId": { ... doctor details ... },
      "slotId": { ... slot details ... },
      "status": "confirmed",
      "appointmentDate": "2026-05-15T09:00:00Z",
      "notes": "Patient has history of allergies",
      "paymentStatus": "paid",
      "videoCallUrl": "https://meet.example.com/room123",
      "createdAt": "2026-04-20T10:00:00Z"
    }
  ],
  "total": 15,
  "page": 1,
  "pages": 2
}
```

10.2 GET APPOINTMENT BY ID

Endpoint: GET /api/appointments/:id
Headers: Authorization: Bearer <token>

Response Format (200):

```
{
  "success": true,
  "appointment": {
    "_id": "507f1f77bcf86cd799439017",
    "patientId": { ... patient details ... },
    "doctorId": { ... doctor details ... },
    "slotId": { ... slot details ... },
    "status": "confirmed",
    "appointmentDate": "2026-05-15T09:00:00Z",
    "notes": "Patient has history of allergies",
    "paymentStatus": "paid",
    "videoCallUrl": "https://meet.example.com/room123"
  }
}
```

10.3 CREATE APPOINTMENT

Endpoint: POST /api/appointments
Headers: Authorization: Bearer <patient_token>

Request Format:

```
{
  "doctorId": "507f1f77bcf86cd799439014",
  "slotId": "507f1f77bcf86cd799439015",
  "notes": "I have chest pain and anxiety"
}
```

Response Format (201):

```
{
  "success": true,
  "message": "Appointment created successfully",
  "appointment": {
    "_id": "507f1f77bcf86cd799439017",
    "patientId": "507f1f77bcf86cd799439011",
    "doctorId": "507f1f77bcf86cd799439014",
    "slotId": "507f1f77bcf86cd799439015",
    "status": "pending",
    "appointmentDate": "2026-05-15T09:00:00Z",
    "notes": "I have chest pain and anxiety",
    "paymentStatus": "unpaid"
  }
}
```

10.4 UPDATE APPOINTMENT

Endpoint: PUT /api/appointments/:id
Headers: Authorization: Bearer <token>

Request Format:

```
{
  "status": "confirmed",
  "notes": "Appointment confirmed. Please arrive 15 minutes early."
}
```

Response Format (200):

```
{
  "success": true,
  "message": "Appointment updated successfully",
  "appointment": { ... updated appointment object ... }
}
```

10.5 CANCEL APPOINTMENT (PATCH)

Endpoint: PATCH /api/appointments/:id
Headers: Authorization: Bearer <token>

Request Format:

```
{
  "status": "cancelled",
  "reason": "Emergency came up"
}
```

Response Format (200):

```
{
  "success": true,
  "message": "Appointment cancelled successfully",
  "appointment": { ... updated appointment object ... }
}
```

10.6 DELETE APPOINTMENT

Endpoint: DELETE /api/appointments/:id
Headers: Authorization: Bearer <token>

Response Format (200):

```
{
  "success": true,
  "message": "Appointment deleted successfully"
}
```

11. PAYMENTS MODULE

The Payments module handles payment processing for appointments.

11.1 GET ALL PAYMENTS

Endpoint: GET /api/payments
Headers: Authorization: Bearer <admin_token>

Query Parameters:

```
?status=completed&userId=507f...&page=1
```

Response Format (200):

```
{
  "success": true,
  "payments": [
    {
      "_id": "507f1f77bcf86cd799439018",
      "appointmentId": "507f1f77bcf86cd799439017",
      "userId": "507f1f77bcf86cd799439011",
      "amount": 50,
      "currency": "USD",
      "status": "completed",
      "paymentMethod": "card",
      "transactionId": "TXN123456789",
      "createdAt": "2026-04-25T15:30:00Z"
    }
  ],
  "total": 100,
  "page": 1,
  "pages": 10
}
```

11.2 GET PAYMENT BY ID

Endpoint: GET /api/payments/:id
Headers: Authorization: Bearer <token>

Response Format (200):

```
{
  "success": true,
  "payment": {
    "_id": "507f1f77bcf86cd799439018",
    "appointmentId": "507f1f77bcf86cd799439017",
    "userId": "507f1f77bcf86cd799439011",
    "amount": 50,
    "currency": "USD",
    "status": "completed",
    "paymentMethod": "card",
    "transactionId": "TXN123456789"
  }
}
```

11.3 INITIATE PAYMENT

Endpoint: POST /api/payments/initiate
Headers: Authorization: Bearer <patient_token>

Request Format:

```
{
  "appointmentId": "507f1f77bcf86cd799439017",
  "amount": 50,
  "paymentMethod": "card"
}
```

Response Format (201):

```
{
  "success": true,
  "message": "Payment initiated successfully",
  "payment": {
    "_id": "507f1f77bcf86cd799439018",
    "appointmentId": "507f1f77bcf86cd799439017",
    "amount": 50,
    "currency": "USD",
    "status": "pending",
    "paymentUrl": "https://payment-gateway.com/pay/123456"
  }
}
```

11.4 UPDATE PAYMENT STATUS

Endpoint: PATCH /api/payments/:id
Headers: Authorization: Bearer <admin_token>

Request Format:

```
{
  "status": "completed",
  "transactionId": "TXN123456789"
}
```

Response Format (200):

```
{
  "success": true,
  "message": "Payment updated successfully",
  "payment": { ... updated payment object ... }
}
```

11.5 CANCEL PAYMENT

Endpoint: DELETE /api/payments/:id
Headers: Authorization: Bearer <token>

Response Format (200):

```
{
  "success": true,
  "message": "Payment cancelled successfully"
}
```

12. REAL-TIME VIDEO CALLS MODULE

This module enables real-time video and audio calls using WebRTC technology with Socket.io for signaling.

12.1 START VIDEO CALL

Endpoint: POST /api/calls/start
Headers: Authorization: Bearer <token>

Request Format:

```
{
  "appointmentId": "507f1f77bcf86cd799439017",
  "callType": "video"  // "audio" or "video"
}
```

Response Format (201):

```
{
  "success": true,
  "message": "Call started successfully",
  "call": {
    "_id": "507f1f77bcf86cd799439019",
    "appointmentId": "507f1f77bcf86cd799439017",
    "initiatorId": "507f1f77bcf86cd799439011",
    "callType": "video",
    "status": "ongoing",
    "startTime": "2026-04-27T14:00:00Z",
    "roomId": "room-507f1f77bcf86cd7"
  }
}
```

12.2 GET CALL STATUS

Endpoint: GET /api/calls/:callId/status
Headers: Authorization: Bearer <token>

Response Format (200):

```
{
  "success": true,
  "call": {
    "_id": "507f1f77bcf86cd799439019",
    "status": "ongoing",
    "participants": [
      {
        "userId": "507f1f77bcf86cd799439011",
        "name": "John Doe",
        "joinedAt": "2026-04-27T14:00:00Z"
      },
      {
        "userId": "507f1f77bcf86cd799439014",
        "name": "Dr. Smith",
        "joinedAt": "2026-04-27T14:01:00Z"
      }
    ],
    "duration": 120
  }
}
```

12.3 END VIDEO CALL

Endpoint: POST /api/calls/:callId/end
Headers: Authorization: Bearer <token>

Request Format:

```
{
  "feedbackRating": 5,
  "feedbackComment": "Great consultation!"
```

```
}
```

Response Format (200):

```
{
  "success": true,
  "message": "Call ended successfully",
  "callData": {
    "_id": "507f1f77bcf86cd799439019",
    "status": "ended",
    "duration": 1200,
    "endTime": "2026-04-27T14:20:00Z"
  }
}
```

12.4 Socket.io EVENTS FOR VIDEO CALLS

Client sends to server:

- join-call: { callId, userId }
- offer: { to, offer }
- answer: { to, answer }
- ice-candidate: { to, candidate }
- toggle-mute: { callId, isMuted }
- toggle-video: { callId, isVideoOn }
- send-message: { to, message }
- end-call: { callId }

Server sends to client:

- offer: { from, offer }
- answer: { from, answer }
- ice-candidate: { from, candidate }
- participant-joined: { userId, name }
- participant-left: { userId }
- call-ended: { reason }
- message-received: { from, message }

13. REAL-TIME CHAT MODULE

The Chat module provides real-time messaging between users with features like typing indicators, read receipts, and message search.

13.1 SEND MESSAGE

Endpoint: POST /api/chat/send
Headers: Authorization: Bearer <token>

Request Format:

```
{
  "recipientId": "507f1f77bcf86cd799439014",
  "message": "Hi Doctor, I have a question about my prescription.",
  "appointmentId": "507f1f77bcf86cd799439017"  // Optional
}
```

Response Format (201):

```
{
  "success": true,
  "message": {
    "_id": "507f1f77bcf86cd799439020",
    "senderId": {
      "_id": "507f1f77bcf86cd799439011",
      "name": "John Doe"
    },
    "recipientId": {
      "_id": "507f1f77bcf86cd799439014",
      "name": "Dr. Smith"
    },
    "message": "Hi Doctor, I have a question...",
    "isRead": false,
    "createdAt": "2026-04-27T14:30:00Z"
  }
}
```

13.2 GET CHAT HISTORY

Endpoint: GET /api/chat/history/:recipientId
Headers: Authorization: Bearer <token>

Query Parameters:

```
?page=1&limit=20&appointmentId=507f...
```

Response Format (200):

```
{
  "success": true,
  "messages": [
    {
      "_id": "507f1f77bcf86cd799439020",
      "senderId": { ... sender details ... },
      "recipientId": { ... recipient details ... },
      "message": "Hi Doctor, I have a question...",
      "isRead": true,
      "readAt": "2026-04-27T14:35:00Z",
      "createdAt": "2026-04-27T14:30:00Z"
    },
    {
      "_id": "507f1f77bcf86cd799439021",
      "senderId": { ... sender details ... },
      "message": "Sure, I'd be happy to help.",
      "isRead": true,
      "createdAt": "2026-04-27T14:32:00Z"
    }
  ],
  "total": 45,
```

```
  "page": 1,
  "pages": 3
}
```

13.3 GET ALL CONVERSATIONS

Endpoint: GET /api/chat/conversations
Headers: Authorization: Bearer <token>

Response Format (200):

```
{
  "success": true,
  "conversations": [
    {
      "conversationPartner": {
        "_id": "507f1f77bcf86cd799439014",
        "name": "Dr. Smith"
      },
      "lastMessage": "Sure, I'd be happy to help.",
      "unreadCount": 2,
      "lastMessageTime": "2026-04-27T14:32:00Z"
    },
    {
      "conversationPartner": {
        "_id": "507f1f77bcf86cd799439022",
        "name": "Dr. Johnson"
      },
      "lastMessage": "See you tomorrow",
      "unreadCount": 0,
      "lastMessageTime": "2026-04-26T10:15:00Z"
    }
  ]
}
```

13.4 MARK MESSAGES AS READ

Endpoint: POST /api/chat/mark-read
Headers: Authorization: Bearer <token>

Request Format:

```
{
  "senderId": "507f1f77bcf86cd799439014"
}
```

Response Format (200):

```
{
  "success": true,
  "message": "Messages marked as read"
}
```

13.5 GET UNREAD COUNT

Endpoint: GET /api/chat/unread-count
Headers: Authorization: Bearer <token>

Response Format (200):

```
{
  "success": true,
  "unreadCount": 5
}
```

13.6 DELETE MESSAGE

Endpoint: DELETE /api/chat/message/:messageId
Headers: Authorization: Bearer <token>

Response Format (200):

```
{
```

```
  "success": true,
  "message": "Message deleted successfully"
}
```

13.7 SEARCH MESSAGES

Endpoint: GET /api/chat/search
Headers: Authorization: Bearer <token>

Query Parameters:

```
?query=prescription&recipientId=507f...
```

Response Format (200):

```
{
  "success": true,
  "results": [
    {
      "_id": "507f1f77bcf86cd799439020",
      "message": "Hi Doctor, I have a question about my prescription.",
      "senderId": { ... },
      "recipientId": { ... },
      "createdAt": "2026-04-27T14:30:00Z"
    }
  ]
}
```

13.8 Socket.io EVENTS FOR CHAT

Client sends to server:

- send-message: { message, recipientId }
- typing: { recipientId }
- stop-typing: { recipientId }
- message-read: { senderId }
- delete-message: { messageId }

Server sends to client:

- new-message: { sender, message, timestamp }
- user-typing: { userId, name }
- user-stopped-typing: { userId }
- messages-read: { count }
- message-deleted: { messageId }
- user-online: { userId }
- user-offline: { userId }

14. ERROR HANDLING & RESPONSE FORMAT

Standard Response Format

Success Response (2xx):

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": { ... response data ... }
}
```

Error Response (4xx/5xx):

```
{
  "success": false,
  "message": "Error description",
  "error": "ERROR_CODE",
  "statusCode": 400
}
```

Common HTTP Status Codes:

• 200 OK - Request successful
• 201 Created - Resource created successfully
• 400 Bad Request - Invalid request parameters
• 401 Unauthorized - Missing or invalid authentication
• 403 Forbidden - Access denied (insufficient permissions)
• 404 Not Found - Resource not found
• 409 Conflict - Resource already exists
• 500 Internal Server Error - Server error

Common Error Examples:

```
// Authentication Error (401)
{
  "success": false,
  "message": "Token is missing or invalid",
  "statusCode": 401
}

// Validation Error (400)
{
  "success": false,
  "message": "Validation failed",
  "errors": {
    "email": "Email is required",
    "password": "Password must be at least 6 characters"
  },
  "statusCode": 400
}

// Resource Not Found (404)
{
  "success": false,
  "message": "Doctor not found",
  "statusCode": 404
}

// Duplicate Resource (409)
{
  "success": false,
  "message": "Email already registered",
  "statusCode": 409
}
```

15. SECURITY BEST PRACTICES

15.1 Authentication & Authorization

• Always send JWT token in Authorization header: "Bearer <token>"
• Tokens expire in 7 days - implement refresh token mechanism
• Hash passwords with bcryptjs before storing in database
• Implement role-based access control (RBAC)
• Validate user permissions on every protected endpoint

15.2 Data Validation

• Validate all input data (type, format, length)
• Sanitize string inputs to prevent injection attacks
• Use email validation regex for email fields
• Validate phone numbers with proper formatting
• Implement file upload restrictions (size, type)

15.3 HTTPS & Encryption

• Always use HTTPS in production
• Encrypt sensitive data at rest
• Use CORS properly to limit cross-origin requests
• Implement Content Security Policy (CSP) headers

15.4 Database Security

• Never expose MongoDB connection strings in code
• Use environment variables for sensitive config
• Implement database indexing for performance
• Regular backups of database
• Use MongoDB Atlas IP whitelisting

15.5 API Security

• Implement rate limiting to prevent DDoS attacks
• Use API keys for third-party integrations
• Log all API requests for audit trail
• Implement request size limits
• Sanitize error messages to avoid information leakage

15.6 Real-time Communication Security

• Verify JWT tokens on Socket.io connections
• Implement user authentication for all socket events
• Use HTTPS/WSS (WebSocket Secure) in production
• Validate all socket event data
• Implement message encryption for sensitive chats

16. API TESTING EXAMPLES

16.1 Using cURL

```
# Register a new user
curl -X POST http://localhost:5000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "name": "John Doe",
    "email": "john@example.com",
    "password": "password123"
  }'

# Login user
curl -X POST http://localhost:5000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "email": "john@example.com",
    "password": "password123"
  }'

# Get all doctors with authentication
curl -X GET http://localhost:5000/api/doctors \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIs..."

# Create appointment
curl -X POST http://localhost:5000/api/appointments \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIs..." \
  -H "Content-Type: application/json" \
  -d '{
    "doctorId": "507f1f77bcf86cd799439014",
    "slotId": "507f1f77bcf86cd799439015",
    "notes": "I have chest pain"
  }'
```

16.2 Using JavaScript/Fetch API

```
// Register user
async function registerUser() {
  const response = await fetch(
    'http://localhost:5000/api/auth/register',
    {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        name: 'John Doe',
        email: 'john@example.com',
        password: 'password123'
      })
    }
  );
  const data = await response.json();
  console.log(data);
}

// Get doctors
async function getDoctors(token) {
  const response = await fetch(
    'http://localhost:5000/api/doctors',
    {
      method: 'GET',
      headers: {
        'Authorization': `Bearer ${token}`
      }
```

```
    }
  );
  const data = await response.json();
  return data;
}

// Send message via Socket.io
const socket = io('http://localhost:5000', {
  auth: { token: 'your_jwt_token' }
});

socket.emit('send-message', {
  recipientId: '507f1f77bcf86cd799439014',
  message: 'Hello Doctor!'
});

socket.on('new-message', (data) => {
  console.log('Message received:', data);
});
```

16.3 Using Python Requests

```
import requests
import json

BASE_URL = 'http://localhost:5000/api'

# Register user
response = requests.post(
    f'{BASE_URL}/auth/register',
    headers={'Content-Type': 'application/json'},
    json={
        'name': 'John Doe',
        'email': 'john@example.com',
        'password': 'password123'
    }
)
print(response.json())

# Login
response = requests.post(
    f'{BASE_URL}/auth/login',
    json={
        'email': 'john@example.com',
        'password': 'password123'
    }
)
token = response.json()['token']

# Get doctors with authentication
headers = {'Authorization': f'Bearer {token}'}
response = requests.get(
    f'{BASE_URL}/doctors',
    headers=headers
)
doctors = response.json()
print(doctors)
```

CONCLUSION & LEARNING POINTS

Key Concepts Covered:

1. RESTful API Design - Creating standardized, predictable endpoints
2. CRUD Operations - Create, Read, Update, Delete resources
3. Authentication - JWT-based stateless authentication
4. Database Design - Schema modeling with relationships
5. Real-time Communication - Socket.io for instant messaging
6. Error Handling - Consistent error response format
7. Security - Best practices for secure API development

Next Steps for Learning:

1. Deploy the application to a cloud platform (Heroku, AWS, GCP)
2. Add unit and integration tests using Jest or Mocha
3. Implement caching layer with Redis
4. Add API documentation with Swagger/OpenAPI
5. Implement comprehensive logging and monitoring
6. Add email notifications and SMS alerts
7. Create admin dashboard for analytics

Resources for Further Learning:

• Express.js Official Documentation: https://expressjs.com
• MongoDB/Mongoose Guide: https://mongoosejs.com
• Socket.io Documentation: https://socket.io/docs
• Node.js Best Practices: https://nodejs.org/en/docs
• RESTful API Design Guide: https://restfulapi.net

Important Reminders:

' Always validate user input
' Use environment variables for configuration
' Implement proper error handling
' Write clean, readable, and documented code
' Test your API endpoints thoroughly
' Follow security best practices
' Keep dependencies updated
' Monitor application performance

This documentation serves as a comprehensive learning resource. Use it as a reference while building your own applications.